



Dotnet France
Technologies Sharepoint, SQL Server & .NET

Association Dotnet France

Notions avancées du langage C#

Version 1.0

U.N.E.M.



Sommaire

1	Introduction.....	4
2	Les notions à savoir	5
2.1	Le type var	5
2.2	Conversions de données	5
2.2.1	Le cast.....	5
2.2.2	Conversion d'une chaîne en nombre	7
2.2.3	Conversion d'un nombre en chaîne	8
2.2.4	Conversion en objet	8
2.3	Le type valeur et le type référence	9
3	Les structures de données.....	11
3.1	Les tableaux.....	11
3.1.1	Déclaration et initialisation	11
3.1.2	Parcourir un tableau à une dimension	11
3.1.3	Dimension multiples.....	12
3.1.4	Parcourir un tableau à dimensions multiples.....	12
3.2	Les Listes.....	13
3.2.1	Déclaration d'une liste	13
3.2.2	Ajouter des éléments à la liste	13
3.2.3	Parcourir la liste.....	14
3.2.4	Action sur la liste	14
3.3	Les énumérations	15
3.3.1	Déclaration et initialisation	15
3.3.2	Utiliser une énumération	15
3.4	Les structures	17
3.4.1	Déclaration et initialisation	17
3.4.2	Utiliser une structure.....	18
4	Les méthodes et gestion des erreurs.....	19
4.1	Création d'une méthode	19
4.1.1	Exemple	19
4.1.2	Passage de paramètres	20
4.1.3	Utilisation des méthodes.....	22



	Gestion des erreurs	22
4.2.1	Importance de la gestion d'exception	22
4.2.2	try	23
4.2.3	catch	24
4.2.4	Finally.....	25
4.2.5	Exception multiples	26
5	Conclusion	28





Introduction

Ce chapitre des notions avancées vient compléter les connaissances acquises dans le précédent chapitre concernant les notions fondamentales du langage c#. Nous allons présenter les notions importantes à savoir, les différentes structures de données et introduire ce que sont les méthodes et la gestion des exceptions.





Les notions à savoir

2.1 Le type var

Dans le Chapitre précédent, nous avons passé en revue les différents types de variables du langage C#. Et nous allons maintenant introduire un nouveau type de variable, le type « var ». Tout d'abord, il faut savoir que le C# est un langage dit à typage « fort ». En effet le C# a un système de type très complet qui nous permet de décrire avec précision le type de notre variable. D'autre part, il y a une sécurité et une « vérification » du typage au moment de la compilation. Cette notion de typage fort fait du C# un langage « sûr ».

Le type var est un type générique qui va nous permettre de stocker tout type de variable, il va donc pouvoir nous permettre de stocker une variable sans préciser son type.

Le type var introduit la notion d'« inférence de type » c'est-à-dire que le compilateur va rechercher le meilleur type pour stocker notre variable ! Au moment de la compilation, le compilateur va analyser notre variable et va la stocker dans le type le plus adapté. Attention toutefois car cela ne marche que pour le cas d'une variable locale c'est-à-dire des variables qui sont déclarées dans une fonction et la variable doit être initialisée au moment de la déclaration.

Exemple :

```
//C#
static void Main(string[] args)
{
    //Déclaration sans typage
    var age1 = 17;
    var age2 = "Quarante ans";
    var age3 = 32.5;

    //Affichage du type des variables
    Console.WriteLine(age1.GetType());
    Console.WriteLine(age2.GetType());
    Console.WriteLine(age3.GetType());
    Console.ReadLine();
}
```

Et la console nous renvoie ceci :

```
System.Int32
System.String
System.Double
```

Ce qui nous prouve bien que le compilateur a sélectionné le type approprié à chaque variable.

2.2 Conversions de données

2.2.1 Le cast

Le cast est une action qui permet de convertir une donnée d'un type vers un autre type. Nous allons voir les différents types de conversion :

2.2.1.1 Le cast implicite

Le cast implicite est un cast qui ne pose aucun problème, car il n'entraîne pas de perte de données. Par exemple si on veut convertir un type «int» en un type «long», la conversion ne posera jamais problème car le type «long» englobe le type «int».

```
//C#  
  
static void Main(string[] args)  
{  
    int age = 42;  
    long longAge = age; //Ici on convertit notre int en long  
  
    Console.WriteLine("Mon âge est : {0}", longAge);  
    Console.ReadLine();  
}
```

Le code compile et la console nous renvoie :

```
Mon âge est 42
```

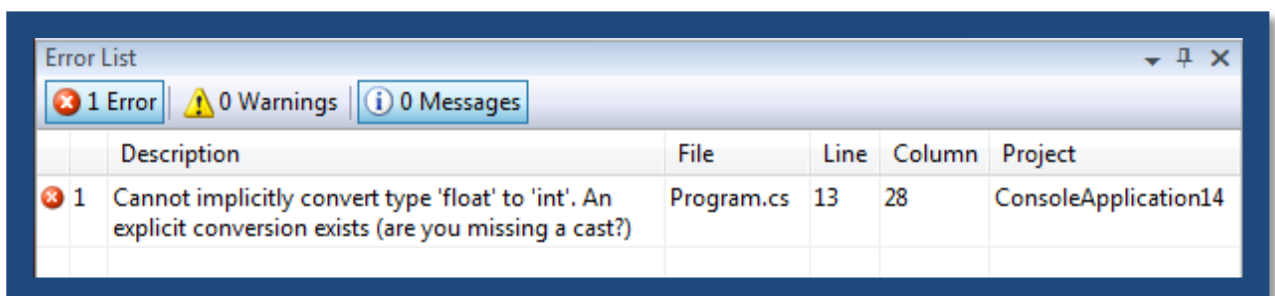
2.2.1.2 Le cast explicite

Tout d'abord nous allons essayer le code suivant et observer ce qu'il se passe.

```
//C#  
  
static void Main(string[] args)  
{  
    float resultatPrecis = 17.83f;  
    int resultat = resultatPrecis;  
  
    Console.WriteLine("Le résultat est : {0}", resultat);  
    Console.ReadLine();  
}
```

Dans le code ci-dessus, on essaie de stocker un résultat à deux chiffres après la virgule dans un entier.

Comme vous pouvez l'imaginer, Visual Studio ne va pas compiler et nous renvoie un joli message d'erreur :





Pour éviter ce message d'erreur et réussir à la compilation, il va nous falloir définir notre conversion de manière explicite. Pour cela, rien de plus simple, on va mettre devant notre variable, le type désiré entre parenthèses, en l'occurrence « int ». Ce qui nous donne :

```
//C#

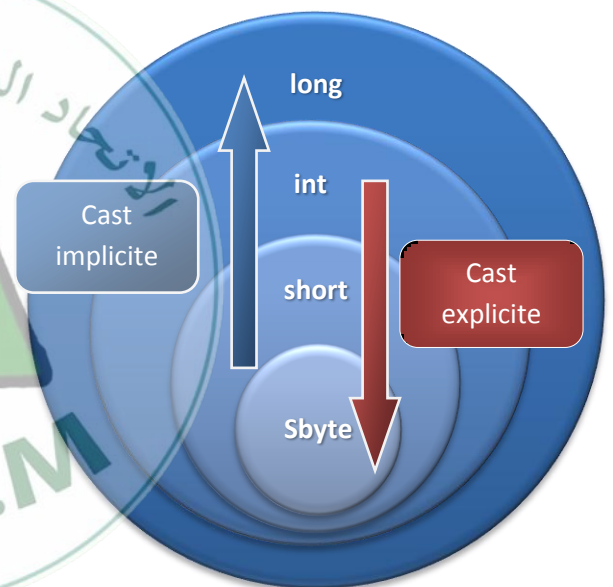
static void Main(string[] args)
{
    float resultatPrecis = 17.83f;
    int resultat = (int)resultatPrecis; //Conversion explicite.

    Console.WriteLine("Le résultat est : {0}", resultat);
    Console.ReadLine();
}
```

Le résultat est : 17

On observe que notre conversion vers les entiers a bien supprimé les chiffres après la virgule, sans effectuer aucun arrondi mais avec un effet de troncature. Il est à noter qu'une conversion explicite peut également se passer sans perte de données.

Le schéma ci-contre nous montre l'imbrication de quelques types de données :



2.2.2 Conversion d'une chaîne en nombre

Ce type de conversion s'avère très utile pour effectuer des opérations sur une chaîne de caractères entrée par l'utilisateur.

Pour convertir une chaîne en nombre on va utiliser la fonction « Parse » précédée du type voulu.

Par exemple si l'on veut convertir « x » en entier on va faire « `int.Parse(x)` » si on le veut en décimal on écrira « `float.Parse(x)` » ou en « long » « `long.Parse(x)` » etc.

Exemple :



```
static void Main(string[] args)
{
    //Deux chaines de caractères :
    string maChaineA = "42";
    string maChaineB = "23";

    //Conversion en entiers puis additon :
    int addition = int.Parse(maChaineA) + int.Parse(maChaineB);

    //Affichage et pause.
    Console.WriteLine(addition);
    Console.ReadLine();
}
```

65

La console nous renvoie 65 nous indiquant que la conversion et l'addition s'est bien déroulée.

2.2.3 Conversion d'un nombre en chaîne

La conversion d'un nombre vers une chaîne de caractère s'effectue grâce à la fonction « ToString() » que l'on va placer à la suite de notre variable à convertir. Pour convertir notre nombre « x » on écrira donc « x.ToString() ».

Exemple :

```
//C#
static void Main(string[] args)
{
    int nombre = 256;

    //Conversion de nombre en chaîne:
    string ChaineNombre = nombre.ToString();

    //Affichage et Pause:
    Console.WriteLine(ChaineNombre);
    Console.ReadLine();
}
```

La conversion se déroule sans soucis et notre chaîne s'affiche :

256

2.2.4 Conversion en objet

La conversion d'une variable en objet est communément appelée « Boxing ». Cette conversion nous permet de pouvoir prendre n'importe quel type de variable et la convertir pour pouvoir la manipuler comme un objet. L'action qui permet de retransformer l'objet en variable s'appelle l'« Unboxing », littéralement le sortir de la boîte.



//C#

```
static void Main(string[] args)
{
    int nombre = 12;

    //Boxing
    object MonObjet = nombre;

    //Unboxing
    int j = (int)MonObjet;
}
```

2.3 Le type valeur et le type référence

En langage C#, il y a deux grands type de données : les données de types valeur et les données de type référence.

Type valeur	Type référence
Tous les types Numériques	Les String
Les caractères	Les tableaux
Les booléens	Les classes
Les structures	Les délégués
Les énumérations	

Exemple de type valeur :

```
//C#
int monEntier = 42;
```

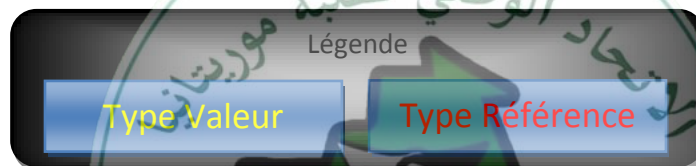
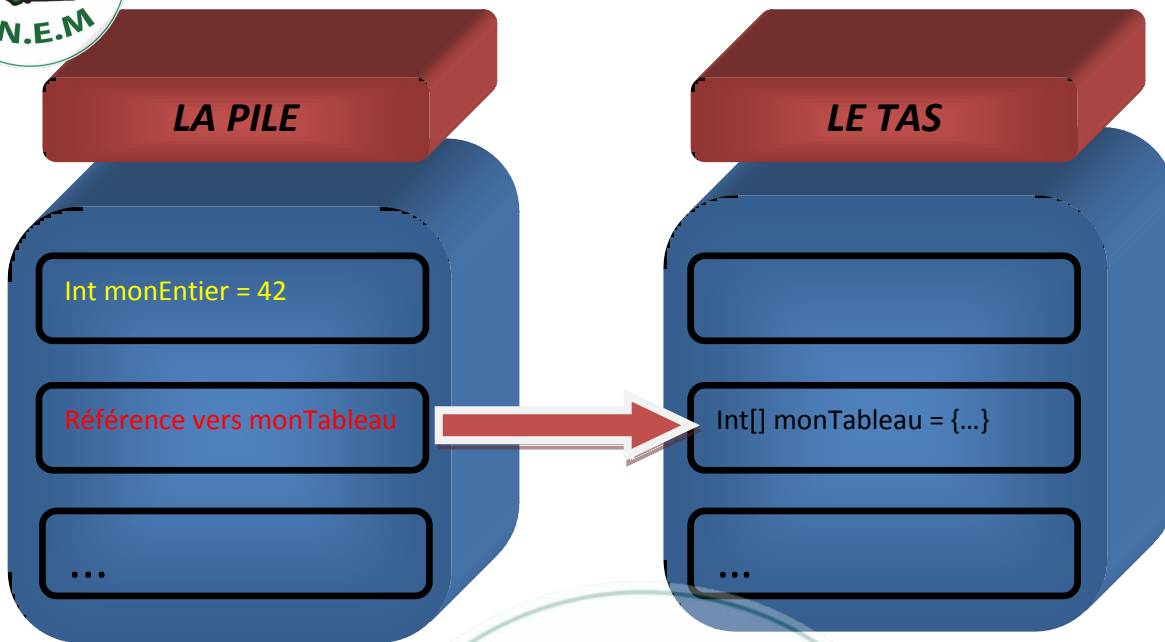
Exemple de type référence :

```
//C#
int[] monTableau = { 2, 4, 6, 8, 10, 11, 12};
```

- Les données de type valeur sont stockés dans la pile.
- Les données de type référence sont stocké dans une partie de la mémoire appelée le tas tandis qu'une référence pointant vers celle-ci se trouve dans la pile.



Le schéma ci-dessous représente la mémoire de la pile et du tas :





Les structures de données

3.1 Les tableaux

3.1.1 Déclaration et initialisation

Un tableau permet de stocker plusieurs variables de même type à la suite dans des cases contiguës de la mémoire. Par exemple si nous déclarons un tableau d'entiers de 7 cases, voici comment ils seront rangés dans la mémoire :

Adresse de la case mémoire	Valeur stockée
1704	345
1705	0
1706	-25
1707	7
1708	42
1709	23
1710	1337

En langage C#, un tableau d'entiers à une seule dimension de 7 cases se déclare comme ceci :

```
//C#
int[] monTableau = new int[7];
```

L'initialisation se déroule comme ceci :

```
//C#
int[] monTableau = new int[] { 345, 0, -25, 7, 42, 23, 1337 };
```

On peut simplifier la syntaxe :

```
//C#
int[] monTableau = { 345, 0, -25, 7, 42, 23, 1337 };
```

On remarque que cette syntaxe ressemble à la syntaxe d'initialisation de variable quelconque, la seule différence étant les crochets devant « `int` » indiquant que c'est un tableau.

3.1.2 Parcourir un tableau à une dimension

On peut parcourir un tableau à une dimension grâce à une boucle for. Cette boucle va afficher la valeur de la case pour chaque rang du tableau. Il ne faut pas oublier qu'un tableau commence au rang 0. Notre indice i va donc être initialisé à zéro et va être incrémentée à chaque tour jusqu'au rang maximum. Afin d'être adaptable à toutes les tailles de tableaux, nous utilisons la méthode « `GetLenght(indice)` » qui renvoie la taille du tableau pour la dimension entre parenthèse.



```

        }
        Console.WriteLine(monTableau[i,j]);
    }
}

```

On peut également parcourir un tableau grâce à une boucle « `foreach` » comme ceci :

```

//C#

foreach (int a in monTableau)
{
    Console.WriteLine(a);
}

```

3.1.3 Dimension multiples

Ici nous allons déclarer un tableau à 2 dimensions. Pour cela il faut rajouter une virgule entre les crochets indiquant que c'est un tableau à 2 dimensions (2 virgules pour un tableau à 3 dimensions, 3 virgules pour un tableau à 4 dimension etc.). Les dimensions sont entre accolades et celles-ci sont séparées entre elles par une virgule, le tout étant lui aussi compris entre accolades.

```

//C#

int[,] monTableau = { { 11, 12, 22, 33, 34, 44,45 } ,
                      { 21, 22, 23, 24, 25, 26,27 } };

```

3.1.4 Parcourir un tableau à dimensions multiples

Ici nous allons appliquer la même méthode que pour un tableau à une seule dimension, sauf que nous allons imbriquer les boucles `for` afin de balayer toutes les dimensions, ici en l'occurrence 2.

```

//C#

int j; //indice de la dimension verticale.
int i; //indice de la dimension horizontale

for (j = 0; j < monTableau.GetLength(0); j++)
{
    for (i = 0; i < monTableau.GetLength(1); i++)
    {
        Console.Write("j={0} i={1} : ",j, i);
        Console.WriteLine(monTableau[j, i]);
    }
}

```

Encore une fois nous pouvons faire la même chose avec une boucle `foreach` assez simple en lui disant d'afficher tout les éléments présents dans notre tableau :



```
foreach (int a in monTableau)
{
    Console.WriteLine(a);
}
```

3.2 Les Listes

Une liste peut être une alternative intéressante au tableau lorsque l'on ne connaît pas le nombre d'éléments à stocker à l'avance.

3.2.1 Déclaration d'une liste

Pour créer une liste rien de plus simple : on crée un objet de type liste grâce au mot clé « `List` » suivi du type d'éléments que l'on souhaite stocker entre crochets :

```
//C#

//Création d'une liste d'entiers :
List<int> MaListe1 = new List<int>();

//Création d'une liste de chaînes :
List<string> MaListe2 = new List<string>();

//...
```

3.2.2 Ajouter des éléments à la liste

A présent notre liste est créée, on va lui ajouter des valeurs, un vrai jeu d'enfant puisque les objets de type liste possède une méthode « `Add()` » qui nous permet d'ajouter des éléments entre parenthèses.

Dans l'exemple suivant, on va rajouter à notre liste les dix premiers nombres de la suite de Fibonacci :

```
//C#

static void Main(string[] args)
{
    List<int> Fibonacci = new List<int>();

    Fibonacci.Add(0);
    Fibonacci.Add(1);
    Fibonacci.Add(1);
    Fibonacci.Add(2);
    Fibonacci.Add(3);
    Fibonacci.Add(5);
    Fibonacci.Add(8);
    Fibonacci.Add(13);
    Fibonacci.Add(21);
    Fibonacci.Add(34);
}
```



3.2.3 Parcourir la liste

A présent nous allons parcourir notre liste et faire afficher chaque élément. Nous allons voir deux méthodes, une « traditionnelle » utilisant la boucle « `for` » et une seconde utilisant une boucle « `foreach` ».

3.2.3.1 Avec une boucle « `for` »

```
//C#

//Boucle "for" traditionnelle :
int i;
for (i = 0; i < Fibonacci.Count(); i++)
{
    Console.WriteLine(Fibonacci[i]);
}
```

3.2.3.2 Avec une boucle « `foreach` »

```
//C#

foreach (int element in Fibonacci)
{
    Console.WriteLine(element);
}
```

Cette boucle signifie « Pour chaque élément dans la liste Fibonacci afficher notre élément ». Le mot « `element` » a été choisi ici par défaut, mais vous pouvez le remplacer par n'importe quel mot de votre choix, à condition d'utiliser le même dans la partie de code de la boucle.

3.2.4 Action sur la liste

Les objets de type « `List` » possèdent de nombreuses méthodes qui nous permettent de modifier notre liste ou bien d'obtenir des informations sur les données stockées dans celles-ci. Nous allons présenter ici quelques exemples de méthodes utiles :

```
//C#

//Méthodes modifiant notre liste :

Fibonacci.Clear(); //Supprime toutes les valeurs
Fibonacci.Insert(place,nb); //Insère notre nombre à la place désirée
Fibonacci.Reverse(); //Range la liste dans le sens inverse.

//Méthode descriptives :

Fibonacci.Count(); //Count retourne le nombre d'éléments
Fibonacci.Max(); //Renvoie la valeur maximum de la liste
Fibonacci.Min(); //Renvoie la valeur minimum de la liste
Fibonacci.Sum(); //Renvoie la somme des éléments
Fibonacci.BinarySearch(nombre); //Renvoie l'indice de l'objet à trouver
Fibonacci.Contains(nombre); //Renvoie vrai si le nombre est trouvé
```



Les énumérations

Une énumération va nous permettre de créer notre propre type de variable afin d'en restreindre les valeurs possibles. Dans l'exemple suivant, nous allons prendre l'exemple des jours de la semaine.

3.3.1 Déclaration et initialisation

Premièrement, nous allons créer l'énumération « Jours » grâce au mot clé « `enum` » (Par convention, le nom d'une énumération est souvent écrit en PascalCase).

```
//C#  
  
enum Jours { lun, mar, mer, jeu, ven, sam, dim };
```

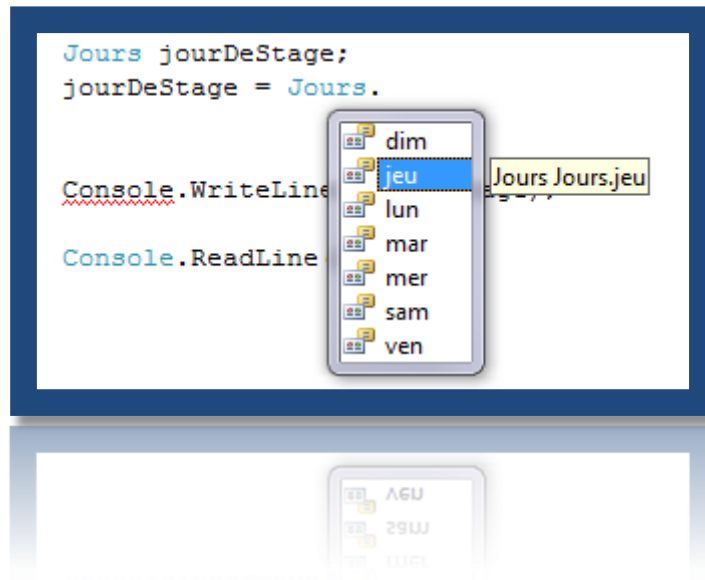
3.3.2 Utiliser une énumération

Ici nous allons poursuivre l'exemple en utilisant l'énumération précédemment déclarée. Dans notre main nous allons créer la variable « jourDeStage » qui sera du type « `Jours` » :

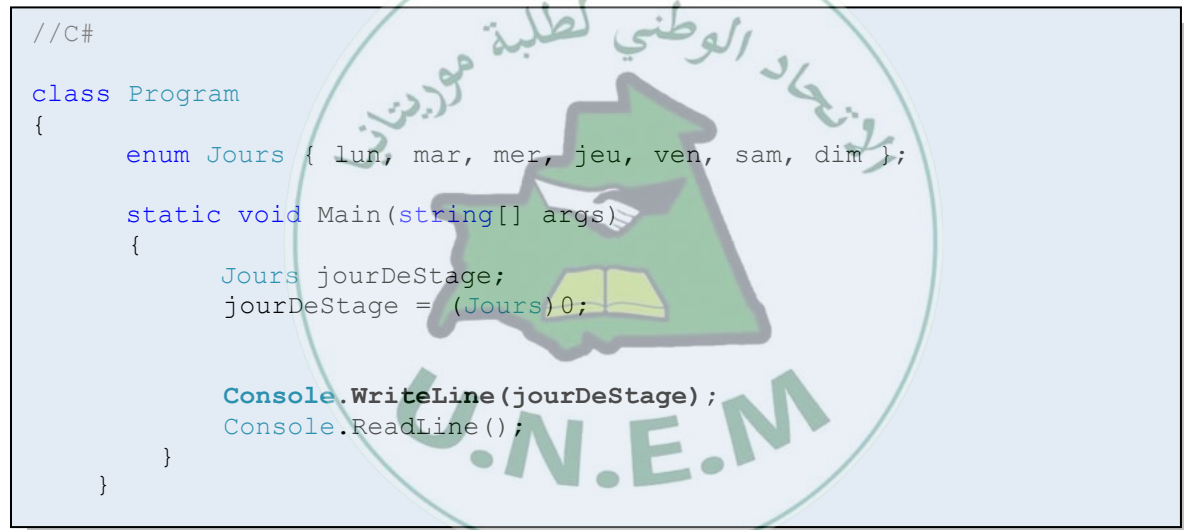
```
//C#  
  
class Program  
{  
    enum Jours { lun, mar, mer, jeu, ven, sam, dim };  
  
    static void Main(string[] args)  
    {  
        Jours jourDeStage; //variable de type "Jours"  
        jourDeStage = Jours.jeu;  
  
        Console.WriteLine(jourDeStage);  
        Console.ReadLine();  
    }  
}
```

Vous avez sûrement remarqué le fait que la déclaration de « `Jours` » se faisait hors de la méthode main. Ce détail est très important puisque la déclaration d'une énumération ne peut s'effectuer à l'intérieur d'une fonction.

Grâce à notre énumération, nous voyons ci-dessous que l'IntelliSense de Visual Studio nous propose les différents choix possibles pour notre variable « jourDeStage ».



Autre remarque importante, les membres d'une énumération peuvent également être appelés avec leur numéro:



Ici, `(Jours)0` équivaut à lundi, `(Jours)1` équivaut à mardi etc. Dernière remarque concernant les énumérations, on peut forcer l'attribution d'un numéro à un membre.



Dans l'exemple suivant nous allons attribuer la valeur 1 à lundi, au lieu de 0, sa valeur par défaut :

```
//C#

class Program
{
    enum Jours { lun=1, mar, mer, jeu, ven, sam, dim };

    static void Main(string[] args)
    {
        Jours jourDeStage;
        jourDeStage = (Jours)3; //équivalent donc à mercredi.

        Console.WriteLine(jourDeStage);
        Console.ReadLine();
    }
}
```

3.4 Les structures

Une structure va nous permettre de rassembler plusieurs types de donnée dans un groupe plus grand. Une structure ressemble un peu à une classe mais une structure est du type valeur à la différence des classes qui sont de type référence.

3.4.1 Déclaration et initialisation

Nous allons prendre l'exemple d'un coureur, un coureur a un nom, un prénom ainsi qu'un numéro de dossard. Nous allons donc créer la structure « Coureur » qui va contenir ses champs. Nous déclarons la structure en dehors du *main* mais nous déclarons les différents champs en « **public** » afin que ceux-ci soient accessibles dans le *main*.

```
//C#

struct Coureur
{
    public string nom;
    public string prénom;
    public int Dossard;
}
```



Utiliser une structure

```
//C#  
  
class Program  
{  
  
    struct Coureur  
    {  
        public string nom;  
        public string prénom;  
        public int Dossard;  
    }  
  
    static void Main(string[] args)  
    {  
        //Déclaration d'une variable de type « coureur » :  
        Coureur lucas;  
  
        //On remplit les champs :  
        lucas.nom = "Dupont";  
        lucas.prénom = "Lucas";  
        lucas.Dossard = 3;  
  
        Console.WriteLine("Nom : " + lucas.nom);  
        Console.WriteLine("Prénom : " + lucas.prénom);  
        Console.WriteLine("Dossard : " + lucas.Dossard);  
  
        Console.ReadLine();  
    }  
}
```

Ici aussi, l'IntelliSense de Visual studio nous proposera les différents champs de la structure lors la saisie du point devant « Lucas ».



Les méthodes et gestion des erreurs

4.1 Création d'une méthode

Une méthode peut être assimilée à ce que l'on appelle dans d'autres langages une « fonction ». Une méthode c'est une boîte dans lequel on va mettre du code. Et pour appeler ce code, on appellera le nom de la boîte !

Notre méthode va se placer à l'extérieur du main (notez que le *main* est en fait la « méthode main », c'est-à-dire la méthode principale du programme, lancée au démarrage de celui-ci).

Une méthode se présente comme ceci :

```
//C#

TypeDeRetour NomDeMaMethode (type parametre)
{
    //Ici on met le code de la méthode
    return variableRetournee;
}
```

Nous allons expliquer un par un les termes ci-dessus :

Le nom de la méthode : On va mettre ici le nom choisi pour notre méthode, nom qui nous servira par la suite à appeler la fonction depuis le main. On choisit ci n'importe quel nom (de préférence explicite) et on l'écrit, par exemple, en PascalCase.

Paramètres : La liste des paramètres est la liste des objets que notre méthode va prendre en entrée. Entre les parenthèses nous allons indiquer le nom de notre paramètre précédé de son type. Prenons l'exemple d'une méthode qui élève un nombre à une puissance donnée. Cette méthode aura par exemple pour paramètre (`int` nombre, `int` puissance).

Le type de retour : c'est le type de la variable que l'on veut renvoyer. Par exemple si on veut une méthode qui calcule le carré d'un nombre qu'on lui envoie. Notre méthode se devra de retourner une variable de type « `int` » contenant le résultat. Notre type de retour sera donc « `int` ». Le type de retour peut être n'importe quel type tel que « `int` » « `long` » « `float` » « `string` » ou il peut être « `void` » indiquant que la méthode ne renvoie aucune valeur. Attention une méthode void ne renvoie pas rien ! Elle renvoie quelque chose puisqu'elle renvoie « void », ce qui signifie « vide ».

Le retour : Lorsqu'on définit une méthode qui renvoie autre chose que void, l'instruction « `return` » est essentielle au renvoi de la donnée indiquée lors de la création de la méthode. Si on crée une méthode qui renvoie un « `int` », il y aura un return suivi d'une variable de type « `int` ».

4.1.1 Exemple

Nous allons créer une méthode qui calcule l'aire d'un rectangle. Notre méthode doit prendre en paramètre une longueur et une largeur, et elle doit nous renvoyer l'aire en valeur entière.



```
static int calculAire (int longueur, int largeur)
{
    int aire = longueur * largeur;
    return aire;
}
```

Pour appeler notre fonction dans le main, rien de plus simple :

```
//C#

//Notre méthode principale "Main"
static void Main(string[] args)
{
    int coteA = 3;
    int coteB = 7;

    //Appel de la méthode :
    Console.WriteLine(CalculAire(coteA, coteB));
    Console.ReadLine();
}
```

4.1.2 Passage de paramètres

Comme vu ci-dessus, les paramètres de la méthode se trouvent entre parenthèses, juste après le nom de celle-ci. Nous allons voir qu'il existe deux moyens de passer des paramètres à une méthode : par valeur et par référence.

4.1.2.1 Par valeur

Lorsque l'on passe des paramètres par valeur, la variable aura toujours sa valeur initiale, il est impossible de la changer à la racine. C'est-à-dire que même si on change cette valeur à l'intérieur d'une méthode, on ne modifiera pas la valeur attribuée lors de l'initialisation du paramètre, mais seulement une copie de cette valeur.

Pour expliquer cette notion assez complexe nous allons nous servir d'un exemple.

```
//C#

//Notre méthode
static int notre_methode(int x)
{
    x = 7;
    return x;
}

//La méthode principale "Main"
static void Main()
{
    int a = 2;

    Console.WriteLine("Avant le passage de la méthode a = " + a);
    Console.WriteLine("Au passage de la méthode\ta = " + notre_methode(a));
    Console.WriteLine("Après le passage de la méthode a = " + a);
    Console.ReadLine();
}
```



La console nous renvoie l'affichage suivant :

```
Avant le passage de la méthode a = 2
Au passage de la méthode      a = 7
Après le passage de la méthode a = 2
```

Au début du programme, notre entier « a » est initialisé à 2 et on le fait afficher par la console qui indique « a=2 » ce qui est tout à fait normal. Puis on fait afficher le retour de notre méthode, celle-ci à pris une copie de « a » et a remplacé sa valeur par 7, la deuxième ligne est donc logique. Après le passage de la méthode, on voit que la valeur de « a » dans le main n'a pas été modifiée.

4.1.2.2 Par référence

Contrairement au type valeur, les paramètres passés en référence sont passés grâce à leur adresse mémoire ce qui permet aux méthodes de modifier directement la valeur à la source. Pour passer un paramètre pas référence, il suffit d'ajouter le mot clé « ref » devant le paramètre, dans la fonction et dans l'appel.

Nous allons reprendre exactement l'exemple précédent, mais cette fois ci avec un passage par référence pour permettre à notre méthode de modifier la valeur de « a ».

```
//C#

//Notre méthode
static int notre_methode(ref int x)
{
    x = 7;
    return x;
}

//La méthode principale "Main"
static void Main()
{
    int a = 2;

    Console.WriteLine("Avant le passage de la méthode a = " + a);
    Console.WriteLine("Au passage de la méthode\ta = " + notre_methode(ref a));
    Console.WriteLine("Après le passage de la méthode a = " + a);
    Console.ReadLine();
}
```

A présent, lorsqu'on exécute on obtient ceci :

```
Avant le passage de la méthode a = 2
Au passage de la méthode      a = 7
Après le passage de la méthode a = 7
```

Et on remarque ici que notre méthode à bien modifié la valeur de « a ».



Utilisation des méthodes

Comme on a pu le voir dans l'exemple précédent, les méthodes sont vraiment utiles, elles nous permettent un important gain de temps et surtout elles sont d'une grande efficacité. D'une part le fait qu'on puisse mettre du code à part à l'intérieur de méthodes et de pouvoir le réutiliser à n'importe quel moment n'importe où dans le programme nous permet aussi une meilleure gestion du programme.

4.2 Gestion des erreurs

4.2.1 Importance de la gestion d'exception

Dans un programme sans gestion des exceptions, certaines actions imprévues peuvent compromettre la fiabilité de notre programme. En effet une simple tentative de division par zéro, l'écriture sur un fichier en lecture seule ou inexistant peut faire tout planter.

Prenons un exemple de programme sans gestion d'exception, dans lequel on demande à l'utilisateur d'entrer son âge pour ensuite l'afficher à la console :

```
//C#  
  
static void Main(string[] args)  
{  
    int age = 0;  
    Console.WriteLine("Quel âge avez-vous?");  
    age = int.Parse(Console.ReadLine());  
    Console.WriteLine("Vous avez " + age + " ans !");  
    Console.ReadLine();  
}
```

Dans ce programme aucune exception n'a été établie. Cela veut donc dire que aucune erreur n'est prise en charge.

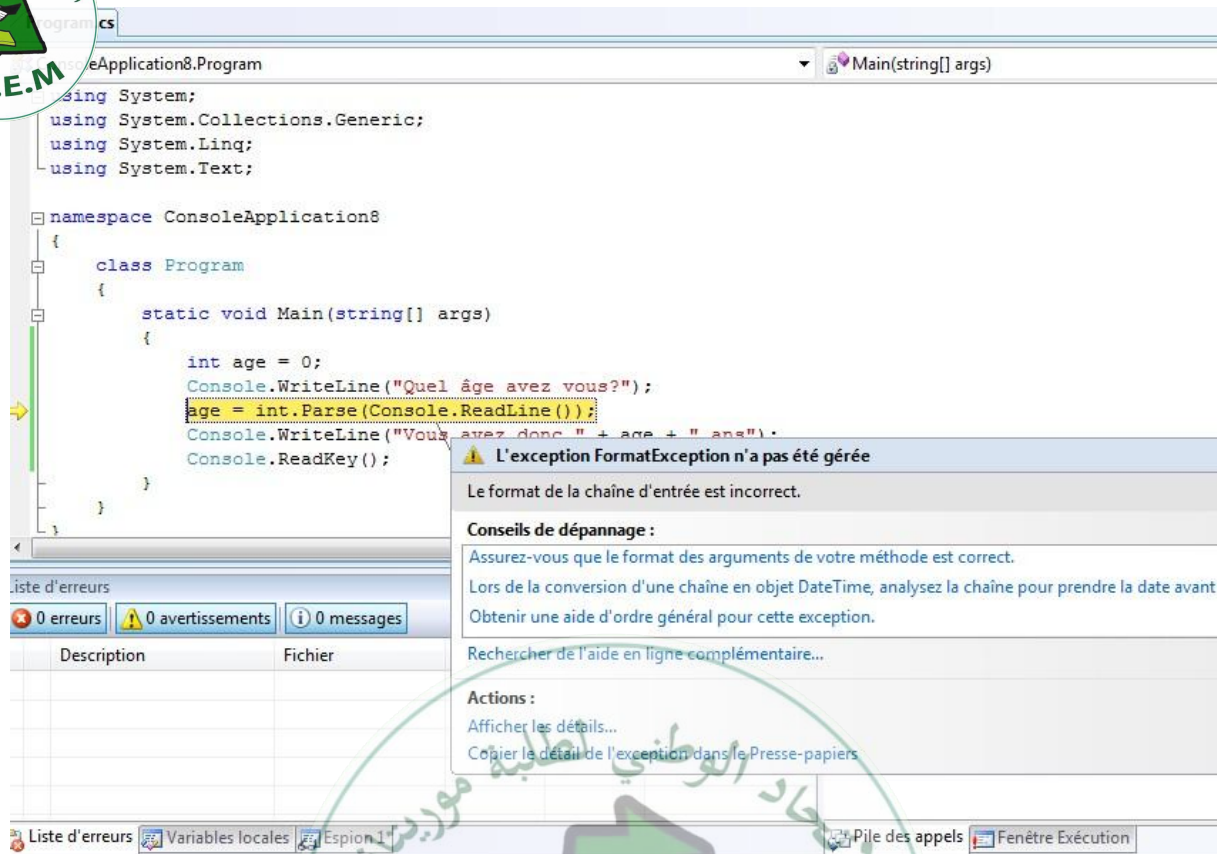
Si l'utilisateur entre un nombre entier, le programme ne pose aucun problème et tout se passe comme prévu :

```
Quel âge avez-vous ?  
20  
Vous avez donc 20 ans
```

Cependant si l'utilisateur entre une chaîne de caractères, quelque chose d'inattendu se produit.

```
Quel âge avez vous ?  
Vingt
```

Le programme s'arrête ici et Visual Studio nous affiche un message d'erreur :



On nous dit que "L'exception FormatException n'a pas été gérée", ce qui nous indique qu'il n'y a aucun contrôle sur l'entrée effectuée par l'utilisateur.

Afin de gérer cette exception, nous allons utiliser les mots clés `try` et `catch` qui formeront deux bloc indispensables. Nous allons maintenant voir de plus près ces mots clés, à quoi servent-ils et comment les utiliser.

4.2.2 try

Le mot « `try` » veut dire « essayer », et c'est dans ce bloc `try` que nous allons placer notre code à risque. Le code placé dans ce bloc va s'exécuter tant qu'aucune erreur ne vient perturber le déroulement du programme.

```
//C#

static void Main(string[] args)
{
    int age = 0;
    Console.WriteLine("Quel âge avez-vous?");

    try
    {
        age = int.Parse(Console.ReadLine());
        Console.WriteLine("Vous avez donc " + age + " ans");
    }

    Console.ReadLine();
}
```



Si on essaie de compiler à cette étape là, Visual Studio nous indique ceci :

Erreurs				
1 erreur 0 avertissements 0 messages				
Description	Fichier	Ligne	Colonne	Projet
1 Catch ou finally attendu	Program.cs	20	13	ConsoleApplication8

En effet il attend de nous un mot clé « catch » ou « finally ». Nous allons maintenant voir le mot clé catch.

4.2.3 catch

Nous avons vu « try » et nous savons qu'il sert à « essayer » du code. Si ce code ne marche pas, par exemple à cause d'une mauvaise entrée de la part de l'interlocuteur, que se passe-t-il ? C'est ici qu'entre en jeu le mot clé catch. Si il y a une erreur, comme dans notre tout premier exemple, au lieu de faire planter le programme le catch va permettre de faire faire au programme une action, par exemple un message d'erreur, qui va indiquer à l'interlocuteur la nature de son erreur. Le catch suit obligatoirement un « try ».

Plaçons maintenant un message d'erreur si l'interlocuteur n'entre pas un nombre entier :

```
//C#
static void Main(string[] args)
{
    int age = 0;
    Console.WriteLine("Quel âge avez-vous?");

    //code à risque
    try
    {
        age = int.Parse(Console.ReadLine());
        Console.WriteLine("Vous avez donc " + age + " ans");
    }

    //Le code placé ici s'exécute si une exception est levée
    catch
    {
        Console.WriteLine("Désolé vous devez écrire un nombre!");
    }
    Console.ReadLine();
}
```

Si on essaie à présent d'écrire une chaîne de caractère voici le message qui sera renvoyé :

```
Quel âge avez-vous ?
Vingt
Désolé vous devez écrire un nombre !
```

Et bien sur le programme ne plante pas. Nous venons de faire une gestion d'exception. Bien que tout cela marche, il reste un mot clé à voir : finally.



Finally

Le bloc finally contient du code qui sera toujours exécuté, qu'il y est une exception levée ou non. Ce bloc est facultatif, il s'utilise comme ceci :

```
//C#

static void Main(string[] args)
{
    int age = 0;
    Console.WriteLine("Quel âge avez-vous?");

    //code à risque
    try
    {
        age = int.Parse(Console.ReadLine());
        Console.WriteLine("Vous avez donc " + age + " ans");
    }

    //Le code placé ici s'exécute si une exception est levée
    catch
    {
        Console.WriteLine("Désolé vous devez écrire un nombre!");
    }
    finally
    {
        Console.WriteLine("\n*** Merci de votre visite ***");
    }
    Console.ReadLine();
}
```

Maintenant si non essayons les deux cas de figure, le « finally » sera toujours exécuté.

Sans exception :

```
Quel âge avez-vous ?
20
Vous avez donc 20 ans

*** Merci de votre visite ***
```

Avec exception :

```
Quel âge avez-vous ?
Vingt
Désolé vous devez écrire un nombre !

*** Merci de votre visite ***
```



Exception multiples

Dans le programme précédant, le même bloc catch « attrapait » toutes les exceptions. Donc notre programme renvoyait le même message d'erreur quel que soit la nature de l'erreur. Il existe un moyen en C# de faire des exceptions multiples, pour cela il suffit de placer plusieurs blocs « catch » en leur associant l'exception appropriée :

```
//C#

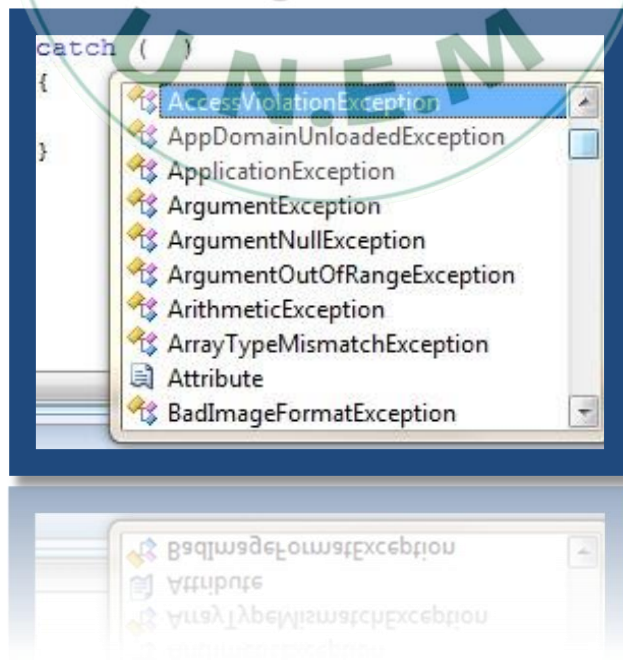
static void Main(string[] args)
{
    try
    {

    }
    catch (Exception)
    {

    }
    catch (Exception)
    {

    }
}
```

On peut bien entendu mettre autant de catch que l'on souhaite. Une différence avec les exceptions multiples c'est que logiquement chaque catch a maintenant une fonction. Cette fonction va être définie entre parenthèses juste après sa déclaration (ici dans l'exemple `Exception`). Visual Studio nous aidera bien à trouver l'exception que nous cherchons en nous proposant la totalité des exception déjà prédéfinies :





Maintenant qu'on a vu son fonctionnement, on va faire un exemple comprenant trois « catch » gérant trois exceptions. Nous allons prendre pour exemple une division, avec trois restrictions : que les nombres soient positifs, pas de divisions par zéro et pas de chaîne de caractères. A chaque bloc catch, nous associerons l'exception associée comme ceci :

```
//C#
static void Main(string[] args)
{
    Console.WriteLine("Nous allons faire une division avec des nombres positifs :");
    Console.Write("\nEnter un premier nombre : ");
    try
    {
        uint a = uint.Parse(Console.ReadLine());
        Console.Write("Enter un deuxieme nombre : ");
        uint b = uint.Parse(Console.ReadLine());
        uint c = a / b;
        Console.WriteLine("Le résultat de la division est " + c);
    }
    catch (OverflowException) //type uint ne peut être négatif
    {
        Console.WriteLine("Enter un nombre positif s'il vous plait");
    }
    catch (DivideByZeroException) //on ne peut diviser par zéro
    {
        Console.WriteLine("Désolé vous ne pouvez pas diviser par zero");
    }
    catch (FormatException) //chaîne de caractères non autorisées car uint
    {
        Console.WriteLine("Les chaîne de caractères ne sont pas autorisées");
    }
    finally
    {
        Console.WriteLine("\n*** Au revoir! ***");
    }
    Console.ReadLine();
}
```



Conclusion

Ce chapitre complète le chapitre précédant concernant les notions fondamentales du langage C#. Nous avons abordé des concepts avancés en passant en revue les notions à savoir tel que le type « var », les types valeur, type référence les conversions de données. Nous avons vu différentes structures de données telles que les tableaux, les listes, les structures et les énumérations. Nous avons terminé ce chapitre en introduisant la notion de méthode puis nous nous sommes intéressés à la gestion des exceptions.

Le prochain chapitre sera un chapitre entièrement dédié aux différents concepts de la programmation objet.

